

DigitalOcean FDE Exercise

Thanks for taking the time to work through this with us. The goal of this project is to give us something concrete to talk about together. The build itself is not the deliverable. The conversation we have when you walk us through it is.

On AI tooling

We are an AI-Native Cloud and we use AI tools heavily in our own work. We expect you will too. Use whatever helps you move faster (Claude Code, Cursor, Gemini, or anything else).

What we care about is that you understand what you built. Be ready to explain every meaningful choice in the design and explain it under questioning. We are not hiring "vibecoders." We are hiring engineers who happen to use AI tools well.

What we are looking for

We are looking for engineers who think clearly about tradeoffs, can explain their choices, and know what they chose not to do. The review session matters as much as the build. We will push on your decisions, so come ready to explain them.

Specifically, we are interested in discussing model selection reasoning, evaluation methodology, cost and production thinking, and how you communicate tradeoffs to a customer.

The project

Build a runnable application that compares language models on a real classification workload, and recommend which model a customer should run in production.

The scenario. A customer is running issue classification at high volume against a frontier model, across many repositories and product lines.. They suspect they are overpaying. Your job is to produce the evaluation that tells them what to actually run in production, with the methodology, numbers, and reasoning to back it up. doctl is one representative slice you will use to build and prove out that evaluation; the recommendation should be one they could carry to the rest of that workload. The review session should sound like a recommendation you would deliver to that customer.

The exercise is about evaluating a range of models and recommending two for production. You will compare those two side by side in the application, but the selection of which two should be the output of broader evaluation work, not a starting assumption.

The dataset. Classify GitHub issues from the [DigitalOcean doctl repository](#) into the label set below. Treat doctl as a proving ground for an approach the customer can later point at other repositories and products, not as a one-off task. doctl has roughly 500 open and closed issues. Some are labeled by maintainers, some are not, and the maintainer labels were applied by different people over years and are not entirely consistent. How you handle that is part of the exercise.

The label set. Categorize each issue into exactly one of these six labels: **bug**, **enhancement**, **question**, **documentation**, **security**, **other**. Use **other** for issues that genuinely do not fit any of the first five (spam, duplicates, off-topic, ambiguous). Treat this as the customer's schema.

Data source

Pull issues from the doctl repository via the GitHub public API. Both open and closed issues are fair game.

This is intentionally a thin ingestion task. We are not testing your ability to build an ingestion pipeline. Spend a few hundred lines on this at most and move on.

The corpus must be stable across runs. How you achieve that is up to you.

Architectural constraints

- The only required external service is the **DigitalOcean Inference Engine (Serverless Inference)** for generation. The SI API is OpenAI-compatible. To get credits for this exercise, create a DigitalOcean team and send us the email address used to create it; we will apply \$200 in credits to that team for you to use here. You will generate your own API key against those credits, and the credits are yours to keep and use however you like afterward. Note that these credits apply only to DigitalOcean-hosted models on Serverless Inference. They will not work against third-party or externally hosted models such as OpenAI or Anthropic, so your comparison is scoped to the open-weight models available on Serverless Inference, and there is no expectation that you benchmark a frontier model directly.
- Everything else is your call. Whatever tools, libraries, or services help you build a clean eval are fine. What matters is that you can explain why you chose what you chose during the review session.

The application

The application is the eval harness. The same code that produces the numbers you walk us through is the code we will run.

Model selection. The application supports comparing two models side by side. The user picks model A and model B from selectors in the UI and runs the comparison against the same corpus. The two models you choose to recommend should be the output of a broader evaluation across more than two candidates; come prepared to discuss what else you considered and why you narrowed to these two.

Per-issue inference. Each issue is classified as its own inference request. Do not batch multiple issues into a single prompt to reduce cost or latency. The customer needs per-call cost, per-call latency, individually-retryable failures, and the ability to route specific cases to fallback models.

Concurrency. The application processes the corpus with parallel inference requests. Concurrency must be configurable without rebuilding the container. Pick a sensible default and be prepared to discuss the tradeoffs.

The corpus, scored and unscored. A subset of the corpus will have ground-truth labels (the evaluation corpus). How you construct that subset, and what you treat as ground truth, is your methodology choice; be prepared to explain your approach. The application classifies the full corpus with both selected models, and presents results in two views: a scored view for the labeled subset, and an unscored view for the rest.

What the UI should show

Scored view (against ground truth):

- Per-model accuracy and per-class precision, recall, F1 (or your chosen equivalents)
- Confusion matrices, side by side
- Drill-down into specific issues where the models disagreed, with the ground-truth label visible

Unscored view (no ground truth):

- Each model's suggested label per issue
- A way to see the raw model output for each item
- Agreement rate between the two models as a headline number
- Per-class distribution of suggestions for each model
- A filter or view for the cases where the models disagreed

Operational metrics, reported per run:

- Cost per call and total cost, with the calculation traceable in your code (we should be able to see how token counts and per-token rates produce the dollar figure)
- p50 and p95 per-request latency, with the concurrency at which they were measured visible alongside them
- Wall-clock time for the full run
- Sustained throughput (requests per second)
- Error rate, broken down by type (rate limit, timeout, other)

Requested scope

1. **Ground truth dataset.** Produce the labeled dataset you will evaluate against. The way you construct your ground truth is a meaningful architectural choice. We do not expect you to hand-label every issue in the set. What matters is that you can articulate the reasoning behind your approach and explain it during the review session.
2. **Model selection.** Evaluate a range of models and recommend two for production. The two you recommend should represent a meaningful tradeoff (capability, cost, latency, parameter count), not minor variants of each other. A 8B model versus a 70B model, or a reasoning model versus a non-reasoning model are all good directions. The selection is part of the exercise; we will ask what else you considered.
3. **Evaluation methodology.** Design the eval. Your conclusions should match the strength of your evidence. We want overall accuracy and per-class performance, confusion matrices or equivalent, and an honest account of where each model fails.
4. **Cost, latency, throughput, and scaling.** The headline numbers (cost per correct classification, p95, throughput, total wall-clock) should be visible in the application.
5. **Production handling.** This exercise proves out a recommendation on a single repository. We are interested in how you would get from there to production across the customer's broader workload: what the rollout looks like, how you would handle the cases where your chosen models fall short, and what would have to be true at each step. We are not asking you to build that system, only to have thought through the path and its failure modes.

Deliverables

A zip file containing:

1. The application source code.
2. A `Dockerfile` that builds a runnable container.
3. A `README` that is primarily a written companion to your work, not a setup guide. It should **include a link to your running application**, and focus on your approach and conclusions: which models you evaluated, which two you are recommending and why, the tradeoffs behind that choice, and what your evaluation actually showed. Include enough on how to build and run the container and what environment variables it expects (including the SI API key) that we could reproduce your run if we wanted to, but keep the weight on your reasoning and recommendation rather than mechanics.
4. Your labeled dataset and any persisted eval results.

A few practical notes

- We are not testing your knowledge of DigitalOcean's platform. We are testing how you think about running production inference workloads. The methodology should port cleanly to any classification problem on any inference provider.
- We do not expect perfection. We expect clear thinking about tradeoffs.
- If you have questions while working on it, please reach out. We would rather answer a question than have you guess at our intent.

Looking forward to seeing what you build.